IBM Innovation Centre for Education

NCS NihonCommunicationSolutions

YENEPOYA (DEEMED TO BE UNIVERSITY)

The Yenepoya institute of arts , science, commerce and management

Project – Low Level Design

on

Natural Language Interface for E-commerce infrastructure

Student Name: Abhishek K K

Industry Mentor:Mr.Shashank **Guided by:Ms.Yuktha**

Table of Contents

1. Introduction 3
1.1 Scope of the Document 3
1.2 Intended Audience 3
1.3 System Overview 3
2. Module Specifications 3
2.1 Flask Backend (app.py) 3
2.2 CLI Frontend (main.py) 4
2.3 Web Frontend Module (app.js) 4
2.3.1 State Object and Persistence 4
2.3.2 Client-Side Router 5
2.3.3 API Integration Functions 5
2.3.4 Auth Module Functions 5
2.3.5 Store and Cart Functions 5
2.3.6 Checkout and Profile Functions 6
2.3.7 UI Utility Functions 6
3. Data Structure Specifications 8
4. Test Specifications 8
5. File Structure and Organization 9
6. Error Handling Strategy 9
7. Logging Specifications 9
8. Deployment Configuration 10
9. Performance Optimization 10
10. Security Implementation 10
11. References 11

1. Introduction

This Low-Level Design document provides detailed implementation specifications for Natural Language Interface for E-commerce infrastructure CLI e-commerce application. It describes internal module structures, function signatures, algorithms, data structures, error handling mechanisms, and comprehensive test specifications for both CLI frontend and Flask backend components.

1.1 Scope of the Document

Covers implementation-level specifications for all three modules: main.py CLI module functions, app.py Flask backend endpoints with algorithms, web frontend app.js function signatures and logic, JSON data structure schemas, test case specifications, file organization, error handling strategies, and deployment configuration details.

1.2 Intended Audience

Python and JavaScript developers implementing or maintaining features, QA engineers writing test cases, code reviewers validating implementation correctness, maintenance teams debugging and enhancing functionality, and technical leads understanding system internals for architecture decisions.

1.3 System Overview

Three primary modules: main.py implements the CLI with menu rendering and flow orchestration; app.py implements the Flask server with four REST endpoints, JSON file I/O utilities, and static/template serving for the web frontend; app.js implements the browser-based SPA with client-side routing, state management, API calls, and all UI rendering logic. CLI and web frontend communicate with the Flask backend over HTTP using JSON payloads. The web frontend additionally persists state to browser localStorage.

2. Module Specifications

2.1 Flask Backend (app.py)

Global Variables: DATA_FILE resolves data.json path relative to script location. Helper Functions: load() opens file, parses JSON, returns dict. save(data) writes dict to file with indent 2. Endpoint GET /products returns data products array. Endpoint GET /cart returns data cart array. Endpoint POST /add validates request.is_json, extracts product and qty from request.json, validates qty positive integer, searches products array, checks stock availability, decrements stock, appends to cart, calls save, returns JSON message. Endpoint GET /checkout checks cart not empty, sets cart to empty array, calls save, returns order confirmation.

2.2 CLI Frontend (main.py)

Function print_main_menu() prints branded header and numbered options. Function get_input(prompt) loops until non-empty input received. Function parse_main_action(choice) maps number or keyword to action string. Function product_flow() calls GET /products, displays table, prompts selection, displays details, prompts add quantity, calls POST /add. Function get_valid_quantity(stock) validates integer input within stock limit. Function cart_flow() calls GET /cart, calculates total, displays formatted summary.Function checkout_flow() calls GET /cart, displays total, prompts confirmation, calls GET /checkout. Function profile_flow() prints static profile details. Function help_flow() lists command aliases. Function main() infinite loop calling print_main_menu, get_input, parse_main_action, route to flow functions with try-except for connection errors.

2.3 Web Frontend Module (static/js/app.js)

2.3.1 State Object and Persistence

Global state object fields: user (object|null, current authenticated user with name/email), cart (array of {product, qty}), wishlist (array of product name strings), products (array, fetched from backend), currentPage (string), searchQuery (string), sortBy (string), categoryFilter (string, default 'all').

Function saveState() — writes state.cart, state.wishlist, and state.user to localStorage under keys ss_cart, ss_wishlist, ss_user. Called after every state mutation.

Function ensureDemoUser() — reads ss_users from localStorage, inserts demo account { name: 'Demo User', email: 'demo@smartstore.com', password: 'demo123' } if not already present. Called on app init.

2.3.2 Client-Side Router

Function navigate(page: string) — removes active CSS class from all .page elements, applies it to #page-{page}, updates state.currentPage, calls window.scrollTo(0,0), and invokes renderStore(), renderCheckout(), or renderProfile() based on target page. Then calls updateNav().

Function updateNav() — reads state.user to determine auth state, toggles display of all elements with data-auth="yes" (shown when authenticated) and data-auth="no" (shown when unauthenticated), updates active class on .nav-link elements.

Function go(page) — alias for navigate(page). Function goHome() — calls navigate('store').

2.3.3 API Integration Functions

Function api(path: string, opts: object) → object|null — async, wraps fetch(BASE + path, { headers: {'Content-Type':'application/json'}, ...opts }), parses JSON, returns null on any exception. BASE constant set to 'http://127.0.0.1:5000'.

Function loadProducts() — async, calls api('/products'), assigns result to state.products, calls renderProductGrid().

2.3.4 Auth Module Functions

Login event listener on #login-form submit — reads email/password inputs, simulates 900ms delay, looks up user in ss_users localStorage array, on match sets state.user and calls saveState() and navigate('store'), on failure calls setError('login-email-error', message).

Signup: Step 1 form collects name/email/password, validates, stores pending user data, shows OTP step (su-step2). Step 2 doSignupStep2() — reads 6 OTP inputs, simulates verification, creates user in ss_users, logs in, navigates to store.

Forgot password: doFpStep1() checks email exists in ss_users. doFpStep2() accepts 6-digit OTP. doFpStep3() validates new password matches confirm field, updates user record in ss_users.

Function demoLogin() — populates login form with demo credentials and submits the form.

Function logout() — sets state.user = null, empties state.cart, removes ss_user and ss_cart from localStorage, calls updateCartBadge(), shows info toast, navigates to login.

2.3.5 Store and Cart Functions

Function renderProductGrid() — reads getFilteredProducts(), iterates to build product card HTML strings. Each card contains: icon from ICONS map, product name, specs + delivery string, price formatted via formatPrice(), stock status with CSS class (low if stock ≤3, out if stock=0), wishlist heart toggle button, and either add-to-cart button or inline qty-control if item already in cart. Injects HTML into #product-grid.

Function getFilteredProducts() → array — applies category map filter, substring search on name/specs, then sort by price-asc/price-desc/name. Returns filtered+sorted product array.

Function addToCart(productName: string) — checks state.user, if null shows toast and navigates to login. Calls api('/add', {method:'POST', body: JSON.stringify({product, qty:1})}). On success pushes {product, qty:1} to state.cart, calls saveState() and updateCartBadge().

Function updateCartQty(productName: string, delta: number) — finds item in state.cart, adds delta to qty. If qty ≤ 0, removes item from cart. Calls saveState(), updateCartBadge().

Function removeFromCart(productName: string) — filters item from state.cart, calls saveState(), updateCartBadge(), renderCartDrawer().

Function cartTotal() → number — reduces state.cart summing (product price × qty) for each item.

Function updateCartBadge() — updates #cart-badge text with state.cart.length, shows/hides badge based on count.

Function renderCartDrawer() — if cart empty shows empty state with Shop Now button. Otherwise maps cart items to HTML rows with qty controls and remove buttons, calculates delivery (free if total > ₹50,000, else ₹99), renders summary and Checkout button.

Function toggleWishlist(productName: string) — adds or removes product name from state.wishlist, calls saveState(), shows toast, calls renderProductGrid().

2.3.6 Checkout and Profile Functions

Function renderCheckout() — guards: if !state.user navigates to login; if cart empty navigates to store with toast. Populates #checkout-items with cart item rows, sets subtotal/delivery/total in summary elements, pre-fills name/email fields from state.user.

Function placeOrder() — disables Place Order button, calls api('/checkout'), waits 800ms, builds order object {id: '#SS'+random4digit, date: toLocaleDateString, items: [...state.cart], total: cartTotal()+delivery, status:'processing'}, unshifts to ss_orders in localStorage, empties state.cart, calls saveState(), shows success div, hides checkout form.

Function renderProfile() — guards: if !state.user navigates to login. Sets avatar initial, display name, email. Reads ss_orders from localStorage, calculates totalSpent. Renders order cards with id, date, item list, total, status badge. Renders wishlist items. Function saveProfile() — reads pf-name/pf-email inputs, updates state.user, calls saveState(), shows success toast.

2.3.7 UI Utility Functions

Function toast(msg: string, type: string) — creates div with class toast {type}, prepends icon based on type (✓/✕/**ℹ**), appends to #toast-container, sets 3000ms timeout to fade-out and remove element.

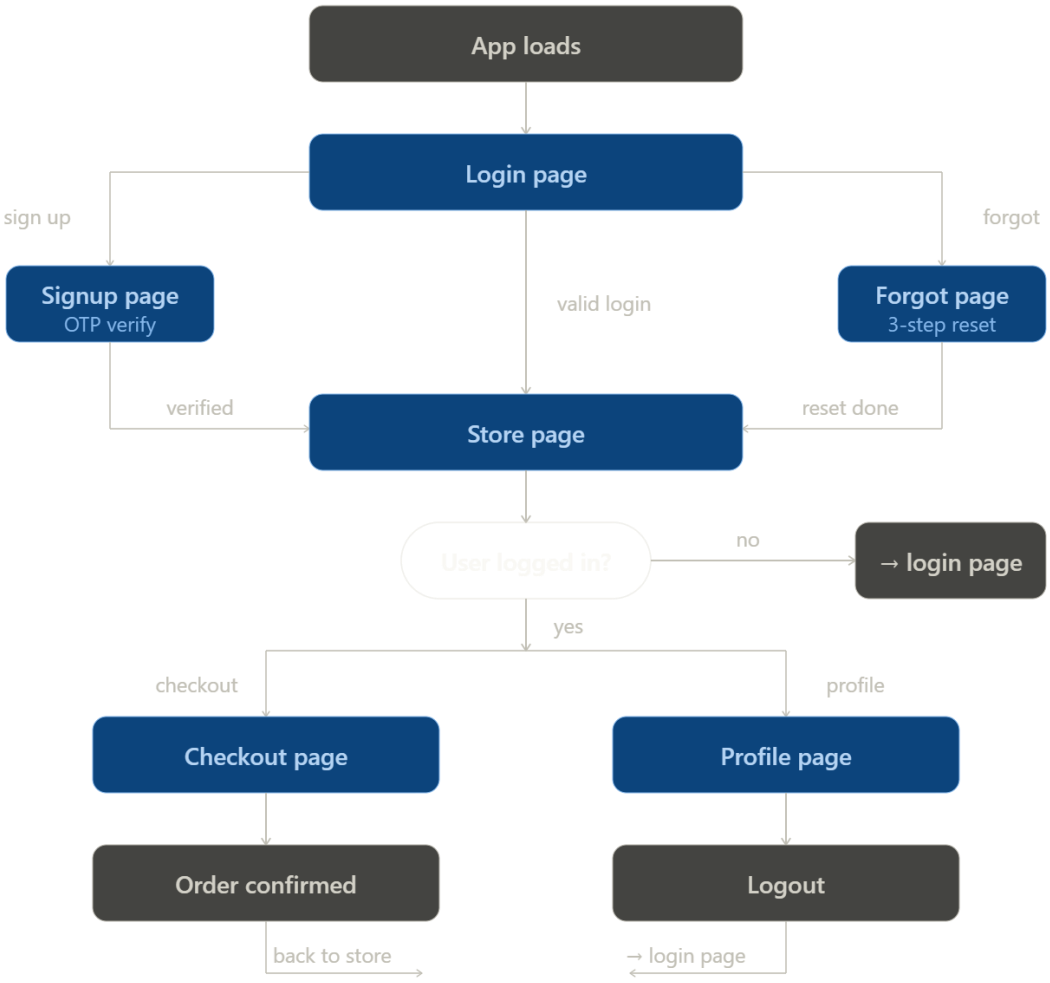
Function formatPrice(n: number) → string — returns Number(n).toLocaleString('en-IN') for Indian number formatting (e.g., 50,000 → 50,000 / 1,00,000).

Function checkStrength(input) — evaluates password length ≥6 (+1), length ≥10 (+1), has uppercase AND digit (+1), applies class weak/medium/strong to seg1/seg2/seg3 elements.

Function toggleEye(button) — toggles adjacent input type between password/text, updates button emoji.

Function openCart() / closeCart() — adds/removes open class on #cart-overlay and #cart-drawer.

Function setCat(button) — removes active from all .cat-tab, sets active on clicked button, updates state.categoryFilter, calls renderProductGrid().


App loads
Login page
sign up
forgot
Signup page
OTP verify
valid login
Forgot page
3-step reset
verified
reset done
Store page
User logged in?
no
→ login page
yes
checkout
profile
Checkout page
Profile page
Order confirmed
Logout
back to store
→ login page

3. Data Structure Specifications

JSON Root Object (data.json): { "products": [...], "cart": [...] }

Product Object: name (string, unique), price (integer, INR), stock (integer, mutable), delivery (string, e.g. "3 days"), specs (string, e.g. "i5 8GB RAM").

Cart Item Object (data.json): product (string, matches product name), qty (integer).

localStorage: ss_users array item: name (string), email (string), password (string, plain text — prototype only).

localStorage: ss_user object: name (string), email (string) — no password stored in session object.

localStorage: ss_cart array item: product (string), qty (integer) — mirrors backend cart.

localStorage: ss_wishlist array: strings of product names.

localStorage: ss_orders array item: id (string, format "#SS" + 4 random digits), date (string, en-IN locale format), items (array of {product, qty}), total (integer, includes delivery charge), status (string: "processing").

ICONS constant (app.js): { Laptop: '💻', Phone: '📱', Headphones: '🎧' } — maps product name to display icon. getIcon(name) returns emoji or '📦' fallback.

.4. Test Specifications

Test navigate('store') renders product grid after loadProducts resolves. Test navigate('checkout') with no user redirects to login. Test navigate('checkout') with empty cart redirects to store. Test addToCart() with no user shows toast and navigates to login. Test addToCart() with valid user calls POST /add and appends item to state.cart. Test updateCartQty(name, -1) on qty=1 item removes item from cart. Test cartTotal() returns correct sum for multi-item cart. Test placeOrder() calls GET /checkout, saves order to ss_orders, clears cart. Test renderProfile() displays correct order count and total spent from ss_orders. Test checkStrength() assigns weak class for 6-char password, strong class for 10+ char with upper and digit. Test formatPrice(100000) returns '1,00,000' (Indian locale). Test login form with valid demo credentials navigates to store. Test login form with invalid credentials shows inline error. Test logout clears state.user, state.cart, and localStorage keys.

5. File Structure and Organization

Project Root: Natural-language-ecommerce/

Primary Backend Module: app.py — Flask server, REST endpoints, template and static file serving.

CLI Frontend Module: main.py — Terminal menu, NLP input parsing, HTTP client.

Web Frontend — Templates: templates/index.html — Single Jinja2 HTML file containing all six page sections.

Web Frontend — Static: static/css/style.css — Dark theme stylesheet with CSS variables, responsive layout, component styles. static/js/app.js — All SPA logic: router, state, auth, API calls, rendering functions.

Data Storage: data.json — Products array and cart array, persistent JSON state.

Documentation: README.md, TEST_INSTRUCTIONS.md, FRONTEND_INSTRUCTIONS.md.

Optional: ecom_env/ (virtual environment), static/.gitignore.

Naming Conventions: snake_case for Python functions/variables, camelCase for JavaScript functions/variables, PascalCase avoided, lowercase for filenames, UPPERCASE for Python constants (DATA_FILE, BASE_URL). JavaScript constants in UPPER_SNAKE (BASE, ICONS).

6. Error Handling Strategy

Web Frontend Error Handling: Function api() wraps all fetch calls in try-catch, returning null on failure. All callers (loadProducts, addToCart, placeOrder) check for null result before processing. Form validation uses setError(id, message) to display inline error messages below input fields. Toast notifications of type 'error' provide feedback for runtime failures. Empty cart guard in renderCheckout() redirects to store rather than showing a broken UI. Auth guards in renderCheckout() and renderProfile() redirect unauthenticated users to login. Password mismatch in forgot-password step 3 is caught client-side with inline error before any state change..

7. Logging Specifications

Web Frontend Feedback (replaces print statements): The toast(msg, type) function serves as the web UI equivalent of CLI print statements. Success operations (add to cart, place order, profile save, login) use type 'success' with ✓ icon. Failures (invalid input, network null, field validation) use type 'error' with ✕ icon. Informational events (logout, wishlist changes, empty cart) use type 'info' with **ℹ** icon. All toasts auto-dismiss after 3 seconds.

8. Deployment Configuration

Prerequisites: Python 3.8 or newer, pip, modern web browser (Chrome/Firefox/Edge recommended).

Dependencies: Flask (backend server + Jinja2 template rendering), requests (CLI HTTP client).

Virtual Environment: python -m venv venv, activate with platform command.

Execution — Backend: python app.py in terminal 1. Flask starts on 127.0.0.1:5000 and serves both the API and the web frontend.

Execution — CLI Interface: python main.py in terminal 2. Connects to backend at BASE_URL = 'http://127.0.0.1:5000'.

Execution — Web Interface: Open browser and navigate to http://127.0.0.1:5000. No additional server needed.

Configuration: Backend BASE URL in app.js set to 'http://127.0.0.1:5000'. Flask DATA_FILE path resolved relative to app.py location. Flask template_folder defaults to templates/, static_folder defaults to static/.

Demo login for web frontend: email demo@smartstore.com, password demo123 (auto-seeded on first load).

9. Performance Optimization

Web Frontend Optimizations: Product grid re-renders operate on in-memory state.products array for search/filter/sort without making additional API calls. Cart badge updates are synchronous DOM mutations with no network round-trip. Animation stagger (60ms per card) applied via CSS animation-delay to avoid rendering all cards simultaneously. localStorage reads are synchronous and fast for the small data volumes used. Heavy operations (placeOrder) show a disabled button with loading text to prevent double submission

10. Security Implementation

Web Frontend Security: Client-side auth with localStorage is prototype-scoped. Passwords stored in plain text in ss_users are unsuitable for production. Checkout and Profile pages implement auth guards that redirect unauthenticated users before rendering. OTP flows are simulated client-side with no actual email dispatch — backend SES integration would be required for production. The "Continue with Google" button is a UI placeholder with no live OAuth backend. Cart manipulation (updateCartQty, removeFromCart) operates on client-side state; backend stock is only decremented via POST /add and is not re-validated on quantity changes made purely client-side.

11. References

Mozilla Developer Network — Fetch API: https://developer.mozilla.org/enUS/docs/Web/API/Fetch_API.
Web Storage API (localStorage) Specification.

Flask Static Files and Jinja2 Template Documentation. JavaScript Single-Page Application Design Patterns.

CSS Custom Properties Specification — W3C.

Number.prototype.toLocaleString() — ECMAScript Specification.